`bisect-ensemble`: A High-Performance Rust Implementation of ReCom for Redistricting Feasibility Analysis

U.10 — Rust Ensemble Engine

Gio Della-Libera

Apportionment Research Group

giodl@microsoft.com

May 2026

Abstract

Markov chain ensemble analysis has become a primary tool for detecting partisan gerrymandering in state-court litigation following *Rucho v. Common Cause* (2019). The dominant implementation, GerryChain (Python), achieves approximately 21–36 steps per second for congressional-scale states (North Carolina: 2,672 tracts, $k = 14$), making a 10,000-step ensemble a 5–8 minute computation. This throughput limits ensemble analysis to an offline, post-hoc step run separately from the redistricting pipeline.

We present `bisect-ensemble`, a Rust implementation of the ReCom Markov chain using Wilson's uniform random spanning tree algorithm [Wilson, 1996]. Wilson's algorithm runs in expected time equal to the cover time of a random walk on the subgraph; for planar graphs this is $O(|V| \log |V|)$ [Aldous, 1990] (the planar bound is Aldous's result, not Wilson's). Census-tract adjacency graphs are planar by construction from non-crossing TIGER shapefiles, so the $O(|V| \log |V|)$ bound applies directly to the redistricting setting. This reduces the per-step cost to native compiled arithmetic at roughly 50,000† steps per second—an estimated 2,300×† speedup over GerryChain

(measured without pair reselection). At this throughput, a 10,000-step ensemble for North Carolina would complete in approximately 0.2 seconds†, making ReCom-style comparison cheap enough for the same workflow family as `bisect state` search modes and audit reports rather than an overnight computation.

The implementation consists of three modules: `spanning.rs` (Wilson's loop-erased random walk), `recom.rs` (the ReCom proposal with balanced-cut enumeration and pair-reselection diagnostics), and `chain.rs` (parallel runner with Rayon, per-chain SHA-256 seeds, $\hat{R}$, ESS, and cut-fraction autocorrelation diagnostics). Benchmark validation against `criterion.rs` is planned for Phase 2 († all throughput and speedup figures are estimates pending `criterion.rs` benchmarks; see Section 5.5).

Contents

1 Introduction **4**

1.1 The Ensemble Bottleneck 4

1.2 The Rust Solution 4

1.3 Pipeline Integration 5

1.4 Paper Organization 5

2 Related Work **6**

2.1 ReCom and GerryChain 6

2.2 Wilson's Spanning Tree Algorithm 6

2.3 Rust for High-Performance Scientific Computing 7

2.4 Portfolio Context 7

3 Algorithm **8**

3.1 Setup and Notation 8

3.2 Wilson's Loop-Erased Random Walk 8

3.3 The ReCom Proposal 9

3.4 Parallel Chains 11

4 Implementation **11**

4.1 Crate Architecture 11

4.2 Graph Representation 12

4.3 Random Number Generation 12
4.4 Balance Check 12
4.5 Serde Output 13
4.6 Texas and Bipartition Failures 13

5 Performance Analysis **14**
5.1 GerryChain Baseline Measurements 14
5.2 Rust Target Estimates 14
5.3 Projected Performance Table 15
5.4 Texas and California 15
5.5 Phase 2: Criterion.rs Benchmarks 16

6 Convergence Diagnostics **17**
6.1 R-hat (Potential Scale Reduction Factor) 18
6.2 Effective Sample Size 19
6.3 Hamming Autocorrelation 20
6.4 Convergence Targets for Publication 20

7 Conclusion **21**
7.1 Summary 21
7.2 Methodological Significance 22
7.3 Future Work 22

1. Introduction

1.1. The Ensemble Bottleneck

Following *Rucho v. Common Cause* [Supreme Court of the United States, 2019], partisan gerrymandering claims have migrated from federal to state courts, and Markov chain ensemble analysis has become the principal expert-witness tool for showing that an enacted map is a statistical outlier. State courts have relied on ensemble evidence in redistricting litigation including *League of Women Voters v. Pennsylvania* (2018), *Harper v. Hall* (2022), and *Harkenrider v. Hochul* (2022). The dominant workflow runs thousands of algorithmically drawn plans using the ReCom chain [DeFord et al., 2021], then places the contested map within the resulting distribution. A plan at the 99th percentile of Republican seat share is characterized as strong evidence of partisan intent; at the 50th percentile, it is indistinguishable from a random draw.

The limiting factor is throughput. The primary open-source implementation, GerryChain [MGGG Redistricting Lab, 2021], is written in Python. For North Carolina (2,672 census tracts, $k = 14$ districts), GerryChain achieves approximately 21 steps per second. A credible 10,000-step ensemble therefore requires roughly 8 minutes; a 50,000-step ensemble requires 40 minutes. At this speed, ensemble analysis is necessarily a separate, overnight computation—a post-hoc audit distinct from the redistricting pipeline itself.

This throughput ceiling creates two practical problems. First, ensemble analysis cannot serve as a *real-time* feasibility check during map-drawing. A redistricting practitioner cannot interactively explore whether a proposed boundary change moves the plan toward or away from the ensemble median. Second, the computational cost discourages the thorough convergence diagnostics—multiple parallel chains, extended runs—that would give researchers and practitioners justified confidence that the marginal distributions of summary statistics have converged across chains (see Della-Libera 2026d).

1.2. The Rust Solution

We present bisect-ensemble, a Rust implementation of the ReCom Markov chain built around Wilson's uniform random spanning tree algorithm [Wilson, 1996]. The design goal is to collapse the throughput gap sufficiently that a 10,000-step ensemble for a congressional-scale

state can become a routine BISECT comparison artifact rather than a separate overnight job.

The key algorithmic innovation is the use of Wilson's loop-erased random walk [Lawler, 1980] to generate uniform random spanning trees. Wilson's algorithm [Wilson, 1996] runs in expected time $O(\tau_{\text{cover}})$, where τ_{cover} is the cover time of the underlying random walk on the subgraph. For planar graphs—the natural setting for census-tract adjacency graphs, which are planar by construction from non-crossing TIGER shapefiles—Aldous [1990] proved that $\tau_{\text{cover}} = O(|V| \log |V|)$. This planar bound is Aldous's result; Wilson's original paper proves the UST correctness and $O(\tau_{\text{cover}})$ cost but does not specialize to planar graphs. Each ReCom step uses the spanning tree of the merged pair region (approximately $2n/k$ nodes, where n is the total tract count and k is the number of districts), so Wilson's cost per step is $O((n/k) \log(n/k))$. Rust's compiled, zero-overhead arithmetic allows this theoretical efficiency to be realized in practice.

1.3. Pipeline Integration

BISECT-ENSEMBLE is implemented today as the Rust ReCom substrate used by BISECT search and sampling code, including the `bisection-ensemble` search mode that runs local two-way ReCom at each bisection node. The standalone `bisect ensemble` command is currently reserved for weighted SMC output; a future ReCom export command should expose the same crate through a user-facing ensemble runner. The current plan-selection surface looks like:

```
bisect state --state NC --year 2020 \
  --search bisection-ensemble \
  --percentile 0.50 --ensemble-steps 200
```

The library output JSON carries chain seeds, step-level cut-fraction and population-deviation records, $\hat{R}$, ESS, and an autocorrelation trace for the cut-fraction proxy. That supports immediate fixed-seed reproduction and summary-metric convergence checks, while the standalone ReCom CLI export remains a follow-on integration item.

1.4. Paper Organization

Section 2 reviews ReCom, Wilson's spanning tree algorithm, and GerryChain. Section 3 gives the formal algorithmic description. Section 4 describes the Rust implementation

architecture. Section 5 presents the performance comparison. Section 6 describes the convergence diagnostics. Section 7 concludes.

2. Related Work

2.1. ReCom and GerryChain

The ReCom (*Recombination*) Markov chain was introduced by DeFord et al. [2021] as a method for sampling redistricting plans with a provably well-defined stationary distribution. At each step, the chain selects a pair of adjacent districts, merges them into a single region, generates a random spanning tree of the merged region using Wilson's algorithm, finds all balanced bipartitions of that tree, and selects one uniformly at random. The chain has a well-defined ReCom sampling distribution over population-balanced redistricting plans; it is not a proof of uniformity over all legally valid plans, and finite runs still require diagnostic checks.

GerryChain [MGGG Redistricting Lab, 2021] is the canonical open-source Python implementation of ReCom, developed and maintained by the MGGG Redistricting Lab at Tufts University. It has been used in expert reports submitted in redistricting litigation in North Carolina [Herschlag et al., 2020], Wisconsin, and Pennsylvania [Chikina et al., 2017], and forms the computational backbone of the ensemble-based auditing literature. GerryChain is designed for correctness and research flexibility over raw throughput; it runs at approximately 21–36 steps per second for congressional-scale states.

Autry et al. [2021] extended the ReCom framework to a multiscale forest variant, improving mixing properties for large k. Carter et al. [2019] analyzed the *merge-split* chain, a closely related Markov chain with distinct stationary distribution properties.

2.2. Wilson's Spanning Tree Algorithm

Wilson [1996] gave the first efficient algorithm for generating uniform random spanning trees of an arbitrary graph. Wilson's algorithm performs a loop-erased random walk from an arbitrary vertex to the already-constructed tree, erasing loops as they form and adding the loop-erased path to the tree. The algorithm runs in expected time equal to the *mean cover time* of a simple random walk on the graph.

The loop-erasing property ensures that the resulting spanning tree is drawn uniformly from the set of all spanning trees of the graph (by Kirchhoff's matrix-tree theorem), making it a direct tool for ReCom proposals. Aldous [1990] proved that the cover time of a random walk on a planar graph with m vertices is $O(m \log m)$, giving Wilson's algorithm $O(m \log m)$ expected time on census-tract adjacency subgraphs.

The loop-erased random walk has a long independent history dating to Lawler [1980], who studied it as a model in statistical mechanics. Propp and Wilson [1998] showed that Wilson's algorithm is a special case of their general *coupling from the past* framework.

2.3. Rust for High-Performance Scientific Computing

Rust's combination of zero-cost abstractions, safe concurrency, and predictable memory layout has made it an increasingly attractive language for computationally intensive scientific applications. The Rayon data-parallelism library [Rayon Contributors, 2024] provides fork-join parallelism with work-stealing scheduling and a threading model that maps cleanly onto independent MCMC chains. Matsakis and Klock [2014] describe the language's ownership system, which eliminates data races by construction—a critical property for parallel MCMC where chains share no mutable state.

Within the `bisect` ecosystem, the Rust codebase has achieved approximately 213× speedup over the Python pipeline for the core METIS partitioning workflow [Della-Libera, 2026a]. The bisect-ensemble crate extends this philosophy to ensemble sampling.

2.4. Portfolio Context

The G-track papers in this portfolio provide the ensemble methodology and comparison results that bisect-ensemble operationalizes. Della-Libera [2026b] establishes the G-track framework for comparing deterministic redistricting algorithms to GerryChain ensembles. Della-Libera [2026c] provides direct percentile placement of the ApportionRegions plans within GerryChain ensembles for six states. Della-Libera [2026d] formalizes the $\hat{R}$, ESS, and Hamming autocorrelation diagnostics implemented in the `bisect` binary. The U.9 paper [Della-Libera, 2026e] establishes the bisection-ensemble connection that motivates local ReCom integration inside the BISECT construction tree. bisect-ensemble narrows a major reproducibility gap: G-track results previously depended on the Python GerryChain installation; the Rust crate makes the core ReCom substrate available inside the BISECT

workspace. Full replacement of the historical G-track runs still requires the Phase 2 cross-validation described in Section 7.

3. Algorithm

3.1. Setup and Notation

Let $G = (V, E)$ be the census-tract adjacency graph for a state with $n = |V|$ tracts and $|E|$ shared boundaries. Each vertex $v \in V$ carries a population $p(v)$. A *k-partition* of G is a function $\sigma : V \to \{1, \dots, k\}$ such that each part $V_i = \sigma^{-1}(i)$ induces a connected subgraph of G. A partition is *population-balanced* if

$$\left| \frac{\sum_{v \in V_i} p(v)}{\bar{p}} - 1 \right| \leq \varepsilon \quad \forall i \in \{1, \dots, k\},$$

where $\bar{p} = \frac{1}{k} \sum_{v \in V} p(v)$ is the target district population and $\varepsilon = 0.005$ is the statutory tolerance.

3.2. Wilson's Loop-Erased Random Walk

Definition 1 (Uniform Random Spanning Tree)**.** *A* uniform random spanning tree *(UST) of a connected graph H is a spanning tree of H drawn uniformly at random from the set of all spanning trees of H.*

Wilson's algorithm [Wilson, 1996] generates a UST of H as follows:

Algorithm 1 Wilson's UST Algorithm

Require: Connected graph $H = (V_H, E_H)$, arbitrary root $r \in V_H$
Ensure: Uniform random spanning tree T of H

```
T ← {r}
while T ≠ V_H do
    Pick any u ∈ V_H \ T
    Perform a simple random walk from u until hitting T, recording the path π
    Erase all loops from π to obtain the loop-erased path LE(π)
    Add all vertices and edges of LE(π) to T
end while
return T
```

Theorem 1 (Wilson 1996; planar bound Aldous 1990)**.** *Algorithm 1 outputs a uniform random spanning tree of H in expected time $O(\tau_{\text{cover}}(H))$, where $\tau_{\text{cover}}(H)$ is the cover time of a simple random walk on H [Wilson, 1996]. For planar graphs with m vertices, Aldous [1990] proved $\tau_{\text{cover}}(H) = O(m \log m)$; this planar specialization is not part of Wilson's original result.*

Planarity of census-tract adjacency subgraphs. In practice, the subgraph $H = G[V_i \cup V_j]$ is a planar graph: it is induced from the census-tract adjacency graph, which is derived from TIGER shapefiles defining non-crossing polygonal boundaries. Small non-planar artefacts (tri-point boundaries, state-line adjacencies) affect at most a small fraction of steps and do not alter the asymptotic cover-time bound.

In our setting, H is the subgraph induced by the union $V_i \cup V_j$ of two adjacent districts chosen for recombination. This subgraph has approximately $m = 2n/k$ vertices, giving an expected Wilson cost of $O((n/k)\log(n/k))$ per ReCom step.

3.3. The ReCom Proposal

Algorithm 2 describes one step of the ReCom chain. The critical design decision is the treatment of tree bipartitions: we enumerate *all* balanced cuts of the spanning tree rather than sampling a single cut. This enumeration is necessary to match the intended ReCom proposal family: the implementation samples a spanning tree, enumerates every balanced tree cut, and then chooses uniformly from the feasible cut set. The paper does not claim that a finite run proves global uniformity over all valid plans.

In the atlas framing, a ReCom step has four inspectable candidate objects: the adjacent district pair, the merged-region spanning tree, the balanced cut set, and the accepted or rejected replacement pair. The chain transcript should distinguish an accepted balanced cut from a no-balanced-cut tree, a pair reselection event, or a slow-moving chain diagnostic. Those are different sampling facts and support different comparison claims.

Failure handling. When a sampled spanning tree T of the merged region R admits no balanced bipartition (i.e., $\mathcal{B} = \emptyset$), the algorithm resamples the entire spanning tree from scratch rather than attempting to repair the same tree. This tree-resample-on-failure policy is essential: retrying with the same tree would not sample a new point from the

Algorithm 2 One ReCom Step

Require: Current partition σ, population tolerance ε
Ensure: Updated partition σ'

$P \leftarrow$ adjacent district pairs in σ, shuffled uniformly
for (i, j) in first 50 entries of P **do**
    $R \leftarrow V_i \cup V_j$ ▷ Merged region
    $H \leftarrow G[R]$ ▷ Induced subgraph
    for $r = 1, \ldots, 10$ **do**
        $T \leftarrow$ WilsonUST(H) ▷ Algorithm 1
        $\mathcal{B} \leftarrow$ all balanced tree cuts of T
        if $\mathcal{B} \neq \emptyset$ **then**
            Sample $e^* \sim \text{Uniform}(\mathcal{B})$
            $\{A, B\} \leftarrow$ components induced by removing e^*
            Update σ' by assigning $V_i \leftarrow A$, $V_j \leftarrow B$
            return σ'
        end if
    end for
end for
return σ ▷ Rejected step after pair-attempt budget

UST distribution and would break the stationary distribution guarantee. If 10 consecutive spanning tree samples for the same district pair all fail to yield a balanced bipartition, the pair is abandoned and a new adjacent pair (i, j) is selected. This pair-reselection mechanism is not a Rust-specific workaround; the same failure mode occurs in Python GerryChain (Section 5 discusses the Texas case in detail). Rust's advantage is simply speed: it exhausts the resample budget faster and moves on.

Stationarity under pair reselection. Pair reselection modifies the ReCom Markov kernel: pairs with low balanced-bipartition feasibility are abandoned more often than high-feasibility pairs, which is not part of the original ReCom specification. Whether this modification preserves the stationary distribution is a conjecture. A plausible argument is that pair reselection is equivalent to a proposal rejection step that discards infeasible pair proposals uniformly—which preserves detailed balance if the pair-selection probability is symmetric over adjacent district pairs and proposals are rejected only on balance failure (never on cut quality). However, this argument has not been formally verified against the specific pair-selection distribution in Algorithm 2. *Stationarity preservation under pair reselection is deferred to Phase 2 empirical verification*: comparing plan-space distributions produced by

BISECT-ENSEMBLE and GERRYCHAIN on the six G-track states will determine whether any distributional difference exceeds statistical resolution.

Balance cut enumeration. For a spanning tree T with $m-1$ edges, each edge removal creates a unique bipartition of the m vertices into two subtrees. Each subtree's population is the sum of $p(v)$ over its vertices. We scan all $m-1$ edges in a single linear pass, accumulating the subtree population on one side via DFS, and classify each edge as balanced or unbalanced. The cost of this enumeration is $O(m)$, subdominant to the $O(m \log m)$ Wilson cost.

3.4. Parallel Chains

Independent chains are embarrassingly parallel: each chain maintains its own partition state, its own random-number generator seeded deterministically from the chain index, and its own step history. No synchronization is required during sampling; chains communicate only at the diagnostics aggregation step.

Per-chain seeds are derived deterministically. The hash input is the fixed-width 32-byte string:

$$\underbrace{\texttt{b"ENSEMBLE\_CHAIN\_"}}_{15\text{ bytes}} \,\|\, \underbrace{\texttt{i.to\_le\_bytes()}}_{8\text{ bytes}} \,\|\, \underbrace{\texttt{b"\_"}}_{1\text{ byte}} \,\|\, \underbrace{\texttt{base\_seed.to\_le\_bytes()}}_{8\text{ bytes}},$$

where i is the zero-indexed chain index and `base_seed` is the user-supplied seed, both encoded as 8-byte little-endian unsigned integers (`u64`). The string literals are ASCII bytes. The total is $15+8+1+8=32$ bytes, giving a fixed-length, unambiguous encoding with no collision risk between chain indices or base-seed values. The first 8 bytes of the SHA-256 digest are interpreted as a little-endian `u64`. The chain runner then calls `SmallRng::seed_from_u64(seed)` from the `rand` crate (version 0.8). This matches the current crate contract exactly and avoids claiming a particular `SmallRng` internal state layout.

4. Implementation

4.1. Crate Architecture

The BISECT-ENSEMBLE crate lives at `crates/bisect-ensemble`. Its core files are:

spanning.rs Wilson's loop-erased random walk. Exports `random_spanning_tree(adj, rng)` returning a parent-array spanning tree. Uses `SmallRng` for per-step randomness (see Section 4.3).

recom.rs The ReCom proposal. Defines `RecomChain::step`, which selects a pair, calls `random_spanning_tree`, enumerates balanced cuts, and records whether the step was accepted. Handles tree-resample-on-failure and pair-reselection logic.

chain.rs Parallel runner. Exports `run_ensemble`, which drives C independent chains via Rayon's `par_iter`, collecting per-step statistics and computing convergence diagnostics.

4.2. Graph Representation

The census-tract adjacency graph is stored as a `Vec<Vec<u32>>` adjacency list. Population data is a `Vec<i64>` aligned to the same vertex indices. Subgraphs for individual district pairs are constructed on demand as `Vec<Vec<u32>>` slices by remapping vertex indices to the compact range $\{0, \ldots, m-1\}$. This subgraph construction cost is $O(m)$ per step and is included in the Phase 2 benchmark contract.

4.3. Random Number Generation

Each chain uses a `SmallRng` instance from the `rand` crate (version 0.8), seeded from the SHA-256-derived per-chain seed as described in Section 3. `SmallRng` is the fast non-cryptographic generator provided by `rand`; its exact internal algorithm is target-dependent, so the paper relies only on the public `rand` 0.8 `seed_from_u64` contract rather than a target-specific state layout. For the random walk within Wilson's algorithm, the dominant operation is selecting a uniformly random neighbor of the current vertex. We avoid modulo bias by using Lemire's fast divisor trick [Lemire, 2019] as implemented in `rand::Rng::gen_range`.

4.4. Balance Check

The balance check for a candidate spanning tree T of the merged region is implemented as a single depth-first traversal. We root T at vertex 0 of the local indexing and compute subtree population sums in $O(m)$ time using a post-order traversal. For each edge $(u, \mathit{parent}(u))$, the subtree rooted at u has population $\mathit{pop}[u]$, and the complementary subtree has population $P_R - \mathit{pop}[u]$, where P_R is the total population of the merged region. The edge is *balanced*

if both subtrees satisfy $|pop/\bar{p} - 1| \leq \varepsilon$. Balanced edges are collected into a vector of local tree-edge pairs and one pair is chosen uniformly when the vector is nonempty.

4.5. Serde Output

Step-level statistics—cut-edge count, normalized cut fraction, population deviation, and accepted/rejected status—are serializable with `serde_json`. Chain-level diagnostics ($\hat{R}$, ESS, and the cut-fraction autocorrelation trace) are stored on the ensemble result. The output schema is versioned: the constant `ensemble_output_version: "1.0"` is emitted as a top-level field, enabling downstream consumers to detect format changes. The current schema is a library result schema; a streaming court-report export is a downstream integration item rather than a completed CLI surface.

4.6. Texas and Bipartition Failures

Texas ($n = 4{,}866$ tracts, $k = 38$ districts) presents a known challenge: the state's shape and high k mean that a randomly merged pair of adjacent districts often contains a region with no balanced bipartition of any random spanning tree. This is a combinatorial property of the Texas plan space, not a Python limitation. GerryChain without pair reselection is documented to fail on Texas from a cold start [MGGG Redistricting Lab, 2021]; GerryChain *with* pair reselection would also handle these cases, albeit more slowly.

bisect-ensemble handles this via the pair-reselection mechanism in Algorithm 2: after 10 consecutive spanning tree resample failures for a given pair, a new adjacent pair is selected. In practice, for Texas, pair reselection triggers frequently during the first 50–100 steps from a cold start, then subsides as the chain moves away from the starting plan into regions of higher feasibility density. bisect-ensemble's advantage over Python GerryChain is throughput: pair reselection at microsecond speed (rather than second speed) reduces warm-up wall time without altering the algorithm's combinatorial behavior. The 10-resample budget is exhausted in microseconds in Rust rather than the seconds it would take in Python, so pair reselection adds negligible wall-time overhead.

The stationarity implications of pair reselection are discussed in Section 3.

5. Performance Analysis

5.1. GerryChain Baseline Measurements

We measured GerryChain throughput by running 1,000 ReCom steps for three states and recording wall-clock time. These are direct empirical measurements from the GerryChain Python package (version 0.3.2, NumPy 1.24, Python 3.10) run on an Intel Core i7-11800H (2.3 GHz base, 4.6 GHz boost, 24 MB L3 cache) under Windows 11. The starting plan for each state is a cold start from a seed-0 random valid plan. No warm-up steps were excluded; all 1,000 steps count toward the wall-clock total. GerryChain was run without pair reselection (the standard configuration):

State	k	n_{tracts}	1K steps (sec)	Steps/sec
North Carolina	14	2,672	47	≈ 21
Wisconsin	8	1,922	28	≈ 36
Georgia	14	1,978	30	≈ 33

The variation in throughput across states reflects differences in both n (tract count) and k (district count). Wisconsin is faster than North Carolina despite having fewer districts ($k = 8$ vs $k = 14$) because its lower tract count reduces the merged-region size; Georgia achieves similar throughput to Wisconsin despite $k = 14$ because its tracts are on average smaller in the adjacency graph.

5.2. Rust Target Estimates

The theoretical throughput of bisect-ensemble follows from Wilson's complexity analysis. For a merged region of size $m = 2n/k$:

1. **Wilson's walk**: $O(m \log m)$ expected operations. At $m \approx 191$ (NC average: $2 \times 2{,}672/14$), Wilson requires approximately $191 \times \log(191) \approx 191 \times 7.5 \approx 1{,}433$ random walk steps.

2. **Effective operations per walk step**: Neighbor selection (1 uniform draw), loop detection (hash set lookup), and path extension (pointer write)—approximately 8 instructions at the clock level.

3. **Total operations per ReCom step**: $\approx 191 \times 8 = 1{,}528$ effective operations.

4. **Effective clock rate**: Modern CPUs execute at $\sim$4 GHz but MCMC-adjacent code (random number generation, irregular memory access) achieves roughly 1 GHz effective throughput on graph-walk workloads of this size.

5. **Resulting time per step**: $1{,}528 \,/\, 10^9 \approx 1.5\,\mu$s.

6. **Steps per second**: $\approx 660{,}000$.

Applying a conservative 13× overhead factor for Rust runtime costs (subgraph construction, adjacency list traversal, SmallVec allocation, serde bookkeeping) yields a practical target of **50,000 steps/sec**. This is still a 2,350× speedup over GerryChain for North Carolina.

Remark 1. *The overhead factor of* 13× *is deliberately conservative. Our METIS pipeline achieved a* 213× *speedup over Python for a structurally similar graph-partitioning workload [Della-Libera, 2026a]. The overhead factor for* bisect-ensemble *is higher because Wilson's random walk has more irregular memory access patterns than METIS's multilevel contraction, but* 50,000 *steps/sec should be achievable without architecture-specific optimization.*

Overhead sensitivity. The 13× overhead factor is an estimate; Table 1 shows how the projected throughput and speedup change under three scenarios.

Table 1: Throughput sensitivity to overhead factor (NC baseline: 21 steps/sec).

Scenario	Overhead factor	Steps/sec	Speedup vs. GerryChain (NC)
Optimistic	6.5×	∼100,000	∼4,700×
Central	13×	∼50,000	∼2,350×
Conservative	26×	∼25,000	∼1,200×

All figures estimated; Phase 2 `criterion.rs` benchmarks will determine the actual overhead factor and narrow the range.

5.3. Projected Performance Table

5.4. Texas and California

The bipartition infeasibility problem in Texas is a combinatorial property of the TX plan space, not a language-specific limitation. GerryChain without pair reselection is documented to fail

Table 2: Performance comparison: GerryChain measured vs. bisect-ensemble estimated. Rust estimates based on Wilson's $O((n/k)\log(n/k))$ analysis at 50,000 steps/sec (conservative). PA and TX estimated from n-scaling of the NC/WI/GA measurements. All Rust figures require validation against `criterion.rs` (Phase 2).

State	k	n	GerryChain (measured)		bisect-ensemble (estimated†)	
			1K steps (s)	Steps/s	10K steps (s)	Speedup
NC	14	2,672	47	21	0.20	2,350×
WI	8	1,922	28	36	0.12	2,330×
GA	14	1,978	30	33	0.13	2,310×
PA	17	3,218	∼30†	∼33	0.18	2,200×
TX	38	4,866	failure‡	—	0.31	—‡
CA	52	8,057	failure‡	—	0.52	—‡

† Rust estimates require validation with `criterion.rs`; see Section 5.5.

‡ GerryChain *without* pair reselection fails on TX ($k = 38$) and CA ($k = 52$) from cold start due to bipartition infeasibility; GerryChain *with* pair reselection and bisect-ensemble both handle these cases via pair reselection (Section 5.4). GerryChain's bipartition failures on TX $k = 38$ are combinatorial, not language-specific. Both GerryChain-with-pair-reselection and bisect-ensemble succeed. The advantage of bisect-ensemble is throughput, not correctness.

on Texas from cold start [MGGG Redistricting Lab, 2021]; GerryChain with pair reselection would also handle these states, albeit more slowly. bisect-ensemble's advantage is that pair reselection at microsecond speed (rather than second speed in Python) reduces warm-up wall time without altering the algorithm's combinatorial behavior. The performance estimate in Table 2 assumes the pair reselection overhead is absorbed within the 50,000 steps/sec budget.

Per-step cost is $O((n/k)\log(n/k))$, so the relevant quantity is the average merged-region size $m \approx 2n/k$. For CA ($k = 52$, $n = 8{,}057$): $m \approx 310$. For PA ($k = 17$, $n = 3{,}218$): $m \approx 379$. PA has the largest average merged region among the six G-track states and therefore the highest per-step cost; California's per-step cost is lower than PA's despite its larger tract count, because its high k reduces the merged-region size. *The actual per-step cost ordering across all six states is deferred to Phase 2 empirical benchmarks.* The estimated 0.52 seconds for 10,000 CA steps and 0.18 seconds for 10,000 PA steps (Table 2) reflect the theoretical ordering; Phase 2 criterion.rs results will confirm or correct the estimates.

5.5. Phase 2: Criterion.rs Benchmarks

All Rust throughput figures in this paper are estimates derived from complexity analysis and are deferred to Phase 2 empirical validation. Phase 2 will validate these estimates using

`criterion.rs`, Rust's standard micro-benchmarking harness, with the following protocol:

Hardware. Benchmarks will be run on the same Intel Core i7-11800H workstation used for the GerryChain baseline measurements in Section 5 (2.3 GHz base, 4.6 GHz boost, 24 MB L3 cache, Windows 11, single-threaded to isolate per-step cost). The CPU model, clock speed, and OS will be reported with results.

GerryChain comparison. The Phase 2 comparison will use GerryChain version 0.3.2, NumPy 1.24, Python 3.10, starting from a cold-start seed-0 random valid plan for each state (matching the Section 5 baseline). Wall-clock time will be measured for 1,000 steps after a 50-step warm-up (the warm-up steps are excluded from the throughput count).

Acceptance criterion. The Phase 2 report will confirm the 50,000 steps/sec estimate if the `criterion.rs` measurement for NC exceeds 30,000 steps/sec (i.e., within 40% of target). If the measurement falls below 30,000 steps/sec, the abstract and Table 2 speedup figures will be revised downward. Results will be reported as 95% confidence intervals from `criterion.rs`, not point estimates.

Granularities. Benchmarks will be run at the following levels:

- **Wilson UST**: Time to generate one uniform random spanning tree for a merged region of size m, as a function of m.
- **Balance check**: Time for the full edge enumeration pass.
- **Full step**: End-to-end ReCom step time including subgraph construction.
- **Chain throughput**: Steps/sec for a 1,000-step chain, compared directly against GerryChain wall-clock time on the same hardware.

Results will be reported in a companion note to this paper.

6. Convergence Diagnostics

The bisect-ensemble chain runner computes three convergence diagnostics in the same pass as the sampling, at zero additional per-step cost beyond tracking per-chain statistics.

The diagnostic framework is described in full in Della-Libera [2026d]; we summarize the implementations here for completeness.

6.1. R-hat (Potential Scale Reduction Factor)

The Gelman-Rubin $\hat{R}$ statistic [Gelman and Rubin, 1992] compares between-chain variance to within-chain variance for a scalar summary statistic θ (e.g., Democratic seat count or Polsby-Popper compactness). For C chains each of length N, let $\bar{\theta}_{c\cdot}$ denote the within-chain mean and $\bar{\theta}_{\cdot\cdot}$ the grand mean. The between-chain variance is

$$B = \frac{N}{C-1} \sum_{c=1}^{C} (\bar{\theta}_{c\cdot} - \bar{\theta}_{\cdot\cdot})^2,$$

and the within-chain variance is

$$W = \frac{1}{C} \sum_{c=1}^{C} s_c^2, \quad s_c^2 = \frac{1}{N-1} \sum_{t=1}^{N} (\theta_{ct} - \bar{\theta}_{c\cdot})^2.$$

The $\hat{R}$ statistic is

$$\hat{R} = \sqrt{\frac{(N-1)/N \cdot W + B/N}{W}}.$$

Values of $\hat{R} < 1.1$ indicate approximate convergence; we target $\hat{R} < 1.05$ for publication-quality results.

Scope of R-hat diagnostics. R-hat diagnostics applied to scalar summary statistics (seat share, compactness) certify convergence of the *marginal distributions* of those statistics across chains, not convergence of the full plan-space distribution. A chain could have $\hat{R} < 1.05$ on Democratic seat share while remaining stuck in a topologically restricted region of plan space. We cite Autry et al. [2021] on multiscale mixing for a comprehensive treatment of the gap between marginal-statistic convergence and plan-space mixing. BISECT-ENSEMBLE's diagnostics are therefore best interpreted as: "the marginal distributions of the reported summary statistics have converged across chains," not as a certification that the full plan-space distribution has been explored.

For redistricting statistics, which are discrete (seat counts take integer values) or bounded (compactness metrics lie in $[0, 1]$), Vehtari et al. [2021] recommend rank-normalised $\hat{R}$. The current BISECT-ENSEMBLE crate computes the classical Gelman-Rubin approximation on

the cut-fraction trace and marks it as a summary-metric diagnostic. A future diagnostics pass should add the rank-normalised variant before using discrete seat-count diagnostics as publication evidence.

Implementation. Each chain accumulates $\bar{\theta}_{c\cdot}$ and s_c^2 incrementally using Welford's online algorithm [Welford, 1962], requiring $O(1)$ storage per chain. In the current crate, $\hat{R}$ is computed in a single pass over the C chain statistics for cut fraction. The G.4 diagnostics validator remains the stronger multi-metric publication surface.

6.2. Effective Sample Size

The effective sample size accounts for autocorrelation within each chain. Using Geyer's initial monotone sequence estimator [Geyer, 1992], the ESS for a single chain of length N is

$$\text{ESS} = \frac{N}{1 + 2\sum_{k=1}^{\hat{K}} \hat{\rho}_k},$$

where $\hat{\rho}_k$ is the lag-k autocorrelation estimate and $\hat{K}$ is the cutoff where the monotone sequence estimate becomes non-positive. For redistricting chains, typical lag-1 autocorrelation is $\hat{\rho}_1 \approx 0.85$–$0.95$, giving ESS $\approx$ 500–2,000 for a 10,000-step chain.

The publication target is the multi-chain ESS formula from Vehtari et al. [2021], which pools within-chain and between-chain information:

$$\text{ESS}_{\text{bulk}} = \frac{CN \cdot \min(\hat{V}/B, 1)}{1 + 2\sum_{k=1}^{\hat{K}} \hat{\rho}_k^{+}},$$

where $\hat{V} = W + B/N$ and $\hat{\rho}_k^{+}$ is the rank-normalised lag-k autocorrelation (computed on the rank-transformed draws as described in Section 6 for $\hat{R}$; see Vehtari et al. 2021 for the full estimator).

The current crate exposes a simpler first-chain ESS estimate using Geyer's initial positive sequence on cut fraction. That estimate is useful as a smoke diagnostic and L1/L2 regression target, but the multi-chain G.4 validator should be used for final publication or litigation tables.

Target. We require ESS ≥ 400 for any summary statistic used in a legal filing. For a 10,000-step, 8-chain run at typical redistricting autocorrelation, the expected ESS is 4,000–16,000—well above this threshold.

6.3. Hamming Autocorrelation

The Hamming distance between two redistricting plans σ and σ' is the fraction of census tracts assigned to different districts:

$$d_H(\sigma, \sigma') = \frac{|\{v : \sigma(v) \neq \sigma'(v)\}|}{n}.$$

The lag-k Hamming autocorrelation is defined using plan-to-plan distances, not reference-based distances:

$$\mathrm{Ham}(k) = \mathrm{Corr}(d_H(\sigma_t, \sigma_{t+k}),\ d_H(\sigma_{t+1}, \sigma_{t+k+1})),$$

where the correlation is taken across steps t. That is, $\mathrm{Ham}(k)$ measures the correlation between consecutive plan-to-plan distances at lag k, not the distance from a fixed reference plan σ_0. In practice, $\hat{\rho}_1 \approx 0.87$–$0.93$ for ReCom, with geometric decay: $\mathrm{Ham}(k) \approx \hat{\rho}_1^k$.

True Hamming autocorrelation serves as a distribution-free mixing diagnostic: it measures how rapidly consecutive plans decorrelate in plan space, independent of any particular summary statistic. A chain with $\mathrm{Ham}(1) > 0.98$ is mixing slowly and may require thinning or a longer run; $\mathrm{Ham}(1) < 0.90$ indicates healthy mixing.

The current crate uses the normalized cut fraction $\phi(\sigma) = |E_\sigma|/|E|$ (the fraction of edges crossing district boundaries) as a computationally efficient running proxy for the autocorrelation trace. $\phi(\sigma)$ is not a collision-free plan identifier: two distinct plans can share the same cut fraction. However, its correlation with plan-to-plan Hamming distance makes it a useful chain-progress diagnostic, and it is the same summary statistic minimized by METIS and reported by the ApportionRegions evaluation.

6.4. Convergence Targets for Publication

Table 3 summarizes the diagnostic thresholds. The target is to achieve all three thresholds after 5,000 steps for all six G-track states. At bisect-ensemble's estimated throughput of 50,000 steps/sec, 5,000 steps require approximately 0.1 seconds.

Table 3: Convergence diagnostic thresholds for BISECT-ENSEMBLE publication results. Current crate fields provide the cut-fraction smoke-diagnostic subset; final publication tables should use the G.4 multi-chain validator.

Diagnostic	Symbol	Target
Gelman-Rubin on reported statistic	$\hat{R}$	< 1.05
Effective sample size	ESS	≥ 400 per statistic
Plan-distance or proxy autocorrelation	Ham(1) / proxy	visible decay

7. Conclusion

7.1. Summary

We have presented BISECT-ENSEMBLE, a Rust implementation of the RECOM Markov chain for redistricting ensemble analysis. The core contribution is the use of Wilson's uniform random spanning tree algorithm, which provides $O((n/k)\log(n/k))$ expected per-step cost, combined with Rust's compiled execution to achieve an estimated 2,300× throughput improvement over GERRYCHAIN.

The practical consequence is a shift in the role of ensemble analysis within the redistricting pipeline. At GERRYCHAIN speeds, a 10,000-step ensemble for North Carolina requires roughly 8 minutes; at BISECT-ENSEMBLE's target throughput, the same ensemble requires approximately 0.2 seconds. This puts ensemble analysis in the same computational tier as compactness scoring and demographic analysis—a routine per-state audit step rather than an overnight computation.

The implementation addresses three correctness requirements that distinguish BISECT-ENSEMBLE from a naive Rust port: (1) full balanced-cut enumeration to match GerryChain's stationary distribution; (2) tree-resample-on-failure rather than same-tree retry; (3) pair-reselection after 10 consecutive tree failures. The TX and CA cold-start challenge is combinatorial, not language-specific: bipartition infeasibility is a property of the plan space geometry. GERRYCHAIN without pair reselection fails on TX/CA from cold start; GERRYCHAIN *with* pair reselection would also handle these cases, albeit more slowly. BISECT-ENSEMBLE's advantage is throughput: Rust reduces the pair-reselection warm-up from seconds to microseconds, making cold-start runs practical within a pipeline budget. Convergence diagnostics ($\hat{R}$, ESS, and a cut-fraction autocorrelation proxy) are computed in-pass for smoke validation and regression testing; final multi-chain publication diagnostics should still flow through the G.4

validator.

7.2. Methodological Significance

BISECT-ENSEMBLE delivers two distinct methodological contributions that should be distinguished:

AEA replication improvement. The G-track papers (G.0–G.5) establish the theoretical and empirical framework for placing APPORTIONREGIONS plans within RECOM ensembles, but their computational results previously depended on a Python GERRYCHAIN installation. BISECT-ENSEMBLE makes the core ReCom substrate reproducible inside the Rust workspace, but the historical G-track claims are not fully replaced until the Phase 2 cross-validation reruns are archived. This is a genuine legal-adjacent contribution: independent reproducibility from deterministic Rust artifacts strengthens the chain-of-custody argument for ensemble results submitted in litigation, provided the output package and validator are included.

Throughput improvement. Longer runs, more chains, and faster convergence diagnostics benefit practitioners and improve internal methodology quality. However, throughput improvement does not alter a court's admissibility determination, which depends on methodological acceptance by the expert-witness community and peer-reviewed validation of the RECOM framework—not on the length of the run. The distinction matters: BISECT-ENSEMBLE's throughput advantage is a contribution to research efficiency, not a legal claim.

7.3. Future Work

Three directions remain for Phase 2:

Benchmark validation (deferred to Phase 2). All throughput figures in this paper are complexity-analysis estimates. `criterion.rs` benchmarks are required to validate the 50,000 steps/sec target and characterize the overhead breakdown (subgraph construction, Wilson walk, balance enumeration, serde output). The full benchmark protocol is specified in Section 5.5.

Pipeline integration. The `bisect-ensemble` crate is already available to the BISECT workspace and to the `bisection-ensemble` search mode. A future standalone ReCom export command should run from a produced plan, emit a versioned ensemble JSON artifact, and report $\hat{R}$, ESS, a plan-distance or proxy autocorrelation trace, and the plan's percentile rank in the resulting ensemble distribution.

Stationarity verification and empirical G-track replication (deferred to Phase 2). The stationarity of pair reselection under the BISECT-ENSEMBLE Markov kernel is a conjecture (Section 3). Once throughput is validated, the six G-track states (NC, WI, GA, PA, TX, CA) should be re-run using BISECT-ENSEMBLE in place of GERRYCHAIN, and the resulting ensemble distributions compared against the G.1 results. Agreement would certify that BISECT-ENSEMBLE's algorithmic choices correctly replicate GERRYCHAIN's stationary distribution and provide empirical evidence that pair reselection does not alter the distributional properties of the chain.

References

David Aldous. The random walk construction of uniform spanning trees and uniform labelled trees. *SIAM Journal on Discrete Mathematics*, 3(4):450–465, 1990. doi: 10.1137/0403039.

Eric A. Autry, Daniel Carter, Gregory J. Herschlag, Zach Hunter, and Jonathan C. Mattingly. Metropolized multiscale forest recombination for redistricting. *Multiscale Modeling & Simulation*, 19(4):1885–1914, 2021. doi: 10.1137/21M1406854.

Daniel Carter, Zach Hunter, Gregory Herschlag, and Jonathan Mattingly. A merge-split proposal for reversible Monte Carlo Markov chain sampling of redistricting plans. *arXiv preprint arXiv:1911.01503*, 2019.

Maria Chikina, Alan Frieze, and Wesley Pegden. Assessing significance in a Markov chain without mixing. *Proceedings of the National Academy of Sciences*, 114(11):2860–2864, 2017. doi: 10.1073/pnas.1617096114.

Daryl DeFord, Moon Duchin, and Justin Solomon. Recombination: A family of markov chains for redistricting. *Harvard Data Science Review*, 3(1), 2021. doi: 10.1162/99608f92.eb30390f.

Gio Della-Libera. From apportionment to boundary design: Extending huntington-hill to congressional redistricting. *Working paper*, 2026a. B.1 — foundational recursive bisection paper.

Gio Della-Libera. Algorithmic redistricting and ensemble methods: A framework for comparison. *Working paper*, 2026b. G.0 — G-track methodology framework.

Gio Della-Libera. Where does the algorithmic map fall? ApportionRegions vs. GerryChain ensemble for six states. *Working paper*, 2026c. G.1 — direct percentile placement.

Gio Della-Libera. Convergence diagnostics for redistricting Markov chains: $\hat{R}$, ESS, and Hamming autocorrelation. *Working paper*, 2026d. G.4 — ensemble diagnostics formalisation.

Gio Della-Libera. Bisection-ensemble duality in algorithmic redistricting. *Working paper*, 2026e. U.9 — bisection-ensemble connection.

Andrew Gelman and Donald B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992. doi: 10.1214/ss/1177011136.

Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992. doi: 10.1214/ss/1177011137.

Gregory Herschlag, Han Sung Kang, Justin Luo, Christy Vaughn Graves, Sachet Bangia, Robert Ravier, and Jonathan C. Mattingly. Quantifying gerrymandering in north carolina. *Statistics and Public Policy*, 7(1):30–38, 2020. doi: 10.1080/2330443X.2020.1828567.

Gregory F. Lawler. A self-avoiding random walk. *Duke Mathematical Journal*, 47(3):655–693, 1980. doi: 10.1215/S0012-7094-80-04741-9.

Daniel Lemire. Fast random integer generation in an interval, 2019.

Nicholas D. Matsakis and Felix S. Klock. The Rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, pages 103–104, 2014. doi: 10.1145/2663171.2663188.

MGGG Redistricting Lab. GerryChain: A python package for redistricting ensembles. `https://github.com/mggg/GerryChain`, 2021. Version 0.3.

James G. Propp and David B. Wilson. How to get a perfectly random sample from a generic Markov chain and generate a random spanning tree of a directed graph. *Journal of Algorithms*, 27(2):170–217, 1998. doi: 10.1006/jagm.1997.0917.

Rayon Contributors. Rayon: A data parallelism library for Rust. `https://github.com/rayon-rs/rayon`, 2024.

Supreme Court of the United States. *Rucho v. Common Cause*, 588 u.s. 684 (2019), 2019.

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. doi: 10.1080/00401706.1962.10490022.

David B. Wilson. Generating random spanning trees more quickly than the cover time. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 296–303. ACM, 1996. doi: 10.1145/237814.237880.